Name: P ARCHITH Roll Number: 2303A52258 Batch: 45

Assignment Number:11.3(Present assignment number)/**24**(Total number of assignments)

Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance

Lab Objectives:

By the end of this lab, students will be able to:

- Design and implement fundamental data structures in Python using AI assistance.
- Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation.
- Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs.
- Improve code readability, efficiency, and maintainability using AI-generated suggestions.

Learning Outcomes

After completing this lab, students will be able to:

- Apply appropriate data structures to solve real-world problems.
- Analyze time and space complexity of different data structure operations.
- Use AI tools responsibly to assist (not replace) logical thinking and problem-solving.

Validate, test, and refine AI-generated code.

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple **Contact Manager Application** to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using **arrays (lists)**.
2. Implement the same functionality using a **linked list** for dynamic memory allocation.
3. Implement the following operations in both approaches:
    - Add a contact
    - Search for a contact
    - Delete a contact
4. Use **GitHub Copilot** to assist in generating search and delete methods.
5. Compare **array vs. linked list** approaches with respect to:
    - Insertion efficiency
    - Deletion efficiency

Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

Prompt: Create a Python program for a Smart Contact Manager.

Requirements:

1. Implement the contact manager using two approaches:

   - Array (Python list) - Linked List

2. Each contact should store:

   - Name - Phone number

3. Implement the following operations in both approaches:

   - Add a contac - Search for a contact by name - Delete a contact by name - Display all contacts

4. For the linked list implementation, create a Node class with name, phone, and next pointer.

5. The program should demonstrate both implementations with sample operations.

6. Use clear class-based structure and simple Python code suitable for data structures learning.

Code:

```python
class ContactManagerArray:
    def __init__(self):
        self.contacts = []
    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print("Contact added")
    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                print("Found:", contact["name"], "-", contact["phone"])
                return
        print("Contact not found")
    def delete_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                self.contacts.remove(contact)
                print("Contact deleted")
                return
        print("Contact not found")
    def display_contacts(self):
        print("\nArray Contacts:")
        for contact in self.contacts:
            print(contact["name"], "-", contact["phone"])
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-26
' 'D:\3-2\AI Assitant Coding\Code Files\Task-11.3.py'
Array Contacts:
Rahul - 9876543210
Anita - 9123456780
Found: Rahul - 9876543210
Contact deleted

Array Contacts:
Anita - 9123456780

LINKED LIST IMPLEMENTATION
Contact added
Contact added

Linked List Contacts:
Rahul - 9876543210
Anita - 9123456780
Found: Anita - 9123456780
Contact deleted

Linked List Contacts:
Anita - 9123456780
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

- Working queue and priority queue implementations.
- Correct prioritization of faculty requests.

Prompt: Create a Python program for a **Library Book Search / Request System**.

Requirements:

1. Implement a **Queue (FIFO)** to manage book borrow requests.
2. Each request should store:
    - Person Name
    - Book Name
    - Role (Student or Faculty)
3. Implement the following methods:
    - enqueue() → add a request
    - dequeue() → process a request
    - display() → show all requests in queue order
4. Extend the program using a **Priority Queue** where:
    - Faculty requests have higher priority than student requests.
    - Faculty requests should be served first.
5. Use Python classes and keep the code simple for Data Structures learning.
6. Demonstrate the system by inserting multiple student and faculty requests and processing them.

Code:

```python
from collections import deque
class LibraryQueue:
    def __init__(self):
        self.queue = deque()
    def enqueue(self, name, book, role):
        request = (name, book, role)
        self.queue.append(request)
        print("Request added:", request)
    def dequeue(self):
        if not self.queue:
            print("No requests in queue")
            return
        request = self.queue.popleft()
        print("Processing request:", request)
    def display(self):
        print("\nQueue Requests:")
        for req in self.queue:
            print(req)
import heapq
class LibraryPriorityQueue:
    def __init__(self):
        self.pq = []
    def enqueue(self, name, book, role):
        # Faculty gets higher priority
        if role.lower() == "faculty":
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-11.3.py'

Queue Requests:
('Kiran', 'Python Programming', 'Student')

PRIORITY QUEUE SYSTEM
Request added: Rahul Data Structures Student
Request added: Anita Machine Learning Faculty
Request added: Kiran Python Programming Student
Request added: Dr. Rao Artificial Intelligence Faculty

Priority Queue Requests:
Anita - Machine Learning - Faculty
Dr. Rao - Artificial Intelligence - Faculty
Kiran - Python Programming - Student
Rahul - Data Structures - Student
Processing: Anita - Machine Learning - Faculty
Processing: Dr. Rao - Artificial Intelligence - Faculty

Priority Queue Requests:
Kiran - Python Programming - Student
Rahul - Data Structures - Student
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a **Last-In, First-Out (LIFO)** approach.

Tasks

1. Implement a **Stack** to manage support tickets.
2. Provide the following operations:
   - push(ticket)
   - pop()
   - peek()
3. Simulate at least **five tickets** being raised and resolved.
4. Use **GitHub Copilot** to suggest additional stack operations such as:
   - Checking whether the stack is empty
   - Checking whether the stack is full (if applicable)

Expected Outcome

- Functional stack-based ticket management system.
- Clear demonstration of LIFO behavior.

Prompt: Create a Python program for an **Emergency Help Desk Ticket System** using a **Stack (LIFO)**.

Requirements:

1. Implement a Stack data structure using a Python list.

2. Each ticket should contain a ticket description or ticket ID.

3. Implement the following stack operations:

* push(ticket) → add a new support ticket

* pop() → resolve the most recent ticket

* peek() → view the most recent ticket without removing it

4. Add additional operations:

* is_empty() → check whether the stack is empty

* is_full() → check whether the stack is full (use a fixed stack size)

* display() → show all tickets in the stack

5. Simulate at least **five support tickets** being raised and resolved to demonstrate **Last-In, First-Out (LIFO)** behavior.

6. Use a class-based implementation suitable for learning Data Structures.

Code:

```python
class HelpDeskStack:
    def __init__(self, size):
        self.stack = []
        self.size = size
    # Check if stack is empty
    def is_empty(self):
        return len(self.stack) == 0
    # Check if stack is full
    def is_full(self):
        return len(self.stack) == self.size
    # Push operation
    def push(self, ticket):
        if self.is_full():
            print("Stack is full. Cannot add ticket.")
        else:
            self.stack.append(ticket)
            print("Ticket added:", ticket)
    # Pop operation
    def pop(self):
        if self.is_empty():
            print("No tickets to resolve.")
        else:
            ticket = self.stack.pop()
            print("Resolved ticket:", ticket)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\C
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-202
' 'D:\3-2\AI Assitant Coding\Code Files\Task-11.3.py'
Ticket added: Ticket 1: WiFi not working
Ticket added: Ticket 2: Printer issue
Ticket added: Ticket 3: Login problem
Ticket added: Ticket 4: Software installation
Ticket added: Ticket 5: Email not syncing

Current Tickets in Stack:
Ticket 5: Email not syncing
Ticket 4: Software installation
Ticket 3: Login problem
Ticket 2: Printer issue
Ticket 1: WiFi not working
Latest ticket: Ticket 5: Email not syncing
Resolved ticket: Ticket 5: Email not syncing
Resolved ticket: Ticket 4: Software installation

Current Tickets in Stack:
Ticket 3: Login problem
Ticket 2: Printer issue
Ticket 1: WiFi not working
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 4: Hash Table

Objective

To implement a **Hash Table** and understand collision handling.

Task Description

Use AI to generate a hash table with:

- Insert
- Search
- Delete

Starter Code

class HashTable:

pass

Expected Outcome

- Collision handling using chaining
- Well-commented methods

Prompt: Create a Python implementation of a **Hash Table with collision handling using chaining**.

Requirements:

1. Create a class named HashTable.
2. Use **separate chaining (linked lists or lists inside buckets)** to handle collisions.

3. Implement the following methods:
    - insert(key, value) → add a key-value pair
    - search(key) → return the value associated with the key
    - delete(key) → remove a key-value pair
4. Implement a simple **hash function using modulo operation**.
5. Initialize the table with a fixed size and create empty buckets.
6. Add clear comments explaining each method.
7. Demonstrate the hash table by inserting several key-value pairs, searching for a key, and deleting a key.

Code:

```
class HashTable:
    def __init__(self, size):
        # Create hash table with empty buckets
        self.size = size
        self.table = [[] for _ in range(size)]
    # Hash function
    def hash_function(self, key):
        return key % self.size
    # Insert method
    def insert(self, key, value):
        index = self.hash_function(key)
        # Check if key already exists
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
        # Add new key-value pair
        self.table[index].append([key, value])
        print("Inserted:", key, value)
    # Search method
    def search(self, key):
        index = self.hash_function(key)
        for pair in self.table[index]:
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-11.3.py'
Inserted: 10 Python
Inserted: 15 Java
Inserted: 20 C++
Inserted: 7 JavaScript

Hash Table:
0 : [[10, 'Python'], [15, 'Java'], [20, 'C++']]
1 : []
2 : [[7, 'JavaScript']]
3 : []
4 : []
Found: [15, 'Java']
Deleted key: 20

Hash Table:
0 : [[10, 'Python'], [15, 'Java']]
1 : []
2 : [[7, 'JavaScript']]
3 : []
4 : []
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 5: Real-Time Application Challenge

Scenario

Design a **Campus Resource Management System** with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in **2–3 sentences**.
3. Implement **one selected feature** using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

Prompt: Design a **Campus Resource Management System** in Python.

Requirements:

1. Identify suitable data structures for the following features:
    - Student Attendance Tracking
    - Event Registration System

- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

2. Provide a mapping table showing:
   Feature → Data Structure → Justification (2–3 sentences).
3. Implement one feature completely in Python.
   Choose **Cafeteria Order Queue** and implement it using a **Queue (FIFO)**.
4. The program should include:
   - place_order(order)
   - serve_order()
   - display_orders()
5. Demonstrate the system by adding and serving multiple orders.

Use a simple class-based Python implementation suitable for Data Structures learning.

Code:

```python
from collections import deque
class CafeteriaQueue:
    def __init__(self):
        self.orders = deque()
    # Place new order
    def place_order(self, order):
        self.orders.append(order)
        print("Order placed:", order)
    # Serve order
    def serve_order(self):
        if not self.orders:
            print("No orders to serve")
        else:
            order = self.orders.popleft()
            print("Serving order:", order)
    # Display all orders
    def display_orders(self):
        print("\nCurrent Orders in Queue:")
        for order in self.orders:
            print(order)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
cripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
' 'D:\3-2\AI Assitant Coding\Code Files\Task-11.3.py'
Order placed: Burger
Order placed: Pizza
Order placed: Sandwich
Order placed: Coffee
Order placed: Pasta

Current Orders in Queue:
Burger
Pizza
Sandwich
Coffee
Pasta
Serving order: Burger
Serving order: Pizza

Current Orders in Queue:
Sandwich
Coffee
Pasta
PS D:\3-2\AI Assitant Coding\Code Files>
```